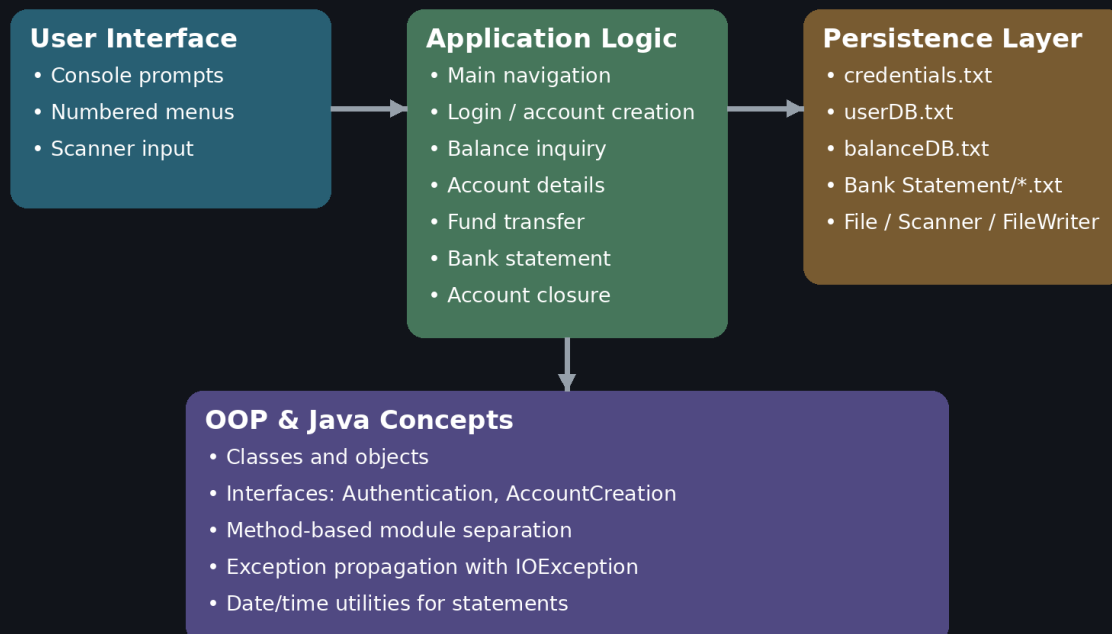


CASE STUDY / SYSTEM DOSSIER

BANK MANAGEMENT SYSTEM BY JAVA

Inside the design, data flow, trade-offs and refactoring opportunities

Layered View of the Current Implementation



Repository under review: <https://github.com/Vedantt-Patel/Bank-Management-System-OOP-JAVA>

01 — The Project in One Minute

This project is a console banking simulation in Java. It turns a familiar banking journey into a compact OOP exercise: a user creates or logs into an account, enters a menu, checks balance and details, transfers money, reads a statement, or closes the account.

Signal	What the repository actually shows
Runtime style	Console application using Scanner and println/printf
Control centre	Main.java
Persistence	Text files under db/
Core data	Credentials, profile, balances, per-account statements
Transaction rule	Receiver must exist; sender must have sufficient balance
OOP markers	Classes plus Authentication and AccountCreation interfaces

Design lens

Think of the repository as a miniature banking service with the database, UI, controller, and domain logic collapsed into one small Java application.

02 — The User Journey

A. Arrival

Main.intro() opens the application and offers two paths: login or create account. An invalid choice loops back to the entry point.

B. Identity

Login reads account number and password, scans the credentials file, and either opens the banking menu, retries for an incorrect password, or retries when the account cannot be found.

C. Banking workspace

Main.menu(accNo) becomes the central router. Each numbered option instantiates the appropriate feature class and passes the current account number.

D. Money movement

Transaction first validates the destination account and then the sender balance. Only after both checks pass does it rewrite the balance file and append debit/credit statement entries.

03 — The Codebase Anatomy

Module	Reads	Writes	Role
Main	-	-	Navigation / orchestration
Login	credentials.txt	-	Authentication
Creation	credentials.txt	credentials.txt, userDB.txt, balanceDB.txt	Provisioning

BalanceInquiry	balanceDB.txt	-	Read current balance
AccountDetails	userDB.txt	-	Read profile
Transaction	balanceDB.txt	balanceDB.txt + statement files	Transfer + audit trail
BankStatement	statement file	-	Present transaction history
Deletion	target text file	same file rewritten	Account closure

04 — Data Flow: A Single Transfer

The transfer path is the most interesting part of the current system because it crosses validation, state mutation and history recording. The sequence is effectively:

Transfer sequence

```

User input
↓
Receiver existence check
↓
Sender balance check
↓
Rewrite balanceDB.txt
↓
Append debit statement
↓
Append credit statement
↓
Return to Main.menu(accNo)

```

This is simple enough to follow in a classroom, but it also exposes an important engineering lesson: multiple file updates can fail independently. A database transaction would eventually be preferable because the balance update and statement writes should succeed or fail as one unit.

05 — OOP: What Is Actually Happening?

The project is object-oriented, but not in the sense of “every banking concept is a rich domain object.” Its main OOP choice is feature-oriented classes. Interfaces create explicit method contracts, while Main composes those classes at runtime.

Pattern	Evidence	Assessment
Abstraction	Authentication / AccountCreation interfaces	Good introductory use
Encapsulation	Feature methods inside dedicated classes	Clear and readable
Composition	Main creates Login, Creation, Transaction, etc.	Strongest architectural pattern here
Inheritance	No central Account hierarchy	Not required by current design
Polymorphism	Minimal explicit use	Opportunity for a future domain model

Learning takeaway

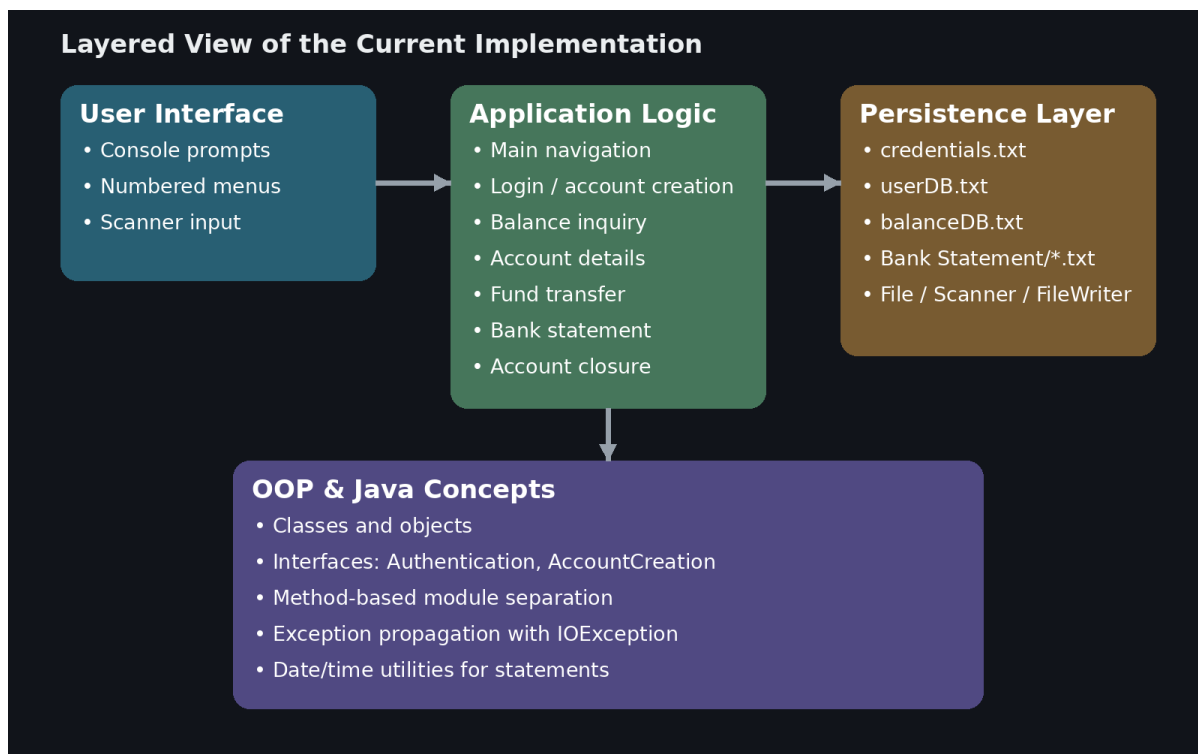
The repository is valuable because the OOP decisions are visible without a framework hiding them. A student can trace a menu option directly into the class that implements it.

06 — Where the Current Design Bends

The first version works because the data model is small. As soon as the application grows, several pressure points appear:

Pressure point	Current choice	Likely consequence
Data integrity	Three separate core text files	Records can drift out of sync
Security	Plaintext credentials / personal data	Unsafe for real-world deployment
Atomicity	Multiple independent FileWriter operations	Partial updates are possible
Input safety	Scanner-driven interactive input	Unexpected formats can break flows
Concurrency	No locking / transaction layer	Concurrent writes are unsafe
Domain model	Primitive strings/ints in procedures	Business rules become harder to evolve
Build structure	Flat Java files in repository	Harder to scale into a standard Java project

07 — A Better Clone: Target Architecture



A cleaner clone can preserve the same feature set while separating presentation, business rules, domain objects and persistence. The first structural move should be to introduce a standard source tree, then isolate file/database access behind repository classes.

Layer	Suggested responsibility	Example target classes
UI	Menus, prompts, output formatting	ConsoleApp, Menu
Service	Business rules and workflows	AccountService, TransferService
Domain	Core banking concepts	Account, Customer, Transaction

Repository	Persistence abstraction	AccountRepository, TransactionRepository
Infrastructure	File or database implementation	FileAccountRepository, SQLiteConnection

08 — Code-to-Feature Traceability

User Feature	Entry Point	Key Operations
Login	Main -> Login.loginFun()	Read credentials, compare values, route user
Create account	Main -> Creation.createAccFun()	Generate ID, collect details, validate, persist
Balance	Main -> BalanceInquiry.balanceInquiryFun()	Find account record, display balance
Details	Main -> AccountDetails.accountDetailsFun()	Find profile, format and display
Transfer	Main -> Transaction.transactionFun()	Validate receiver + balance, mutate balances, log both sides
Statement	Main -> BankStatement.bankStatementFun()	Read statement text, split fields, print table
Close account	Main -> Deletion.accCloseFun()	Confirm and remove account records

09 — Engineering Review

What I would keep

- The menu-driven user journey is straightforward.
- One class per major feature is easy to teach and navigate.
- The transfer workflow has the right high-level business checks.
- Statement files make the result of each transfer observable.
- The interfaces are a useful stepping stone toward cleaner abstractions.

What I would redesign first

- Move from primitive file parsing to domain objects such as Account and Transaction.
- Centralize persistence rather than opening and rewriting files directly in feature classes.
- Hash passwords instead of persisting plaintext credentials.
- Use a single repository/data source so account state is not duplicated across multiple files.
- Replace recursive retry flows with controlled loops and dedicated validation helpers.
- Add automated unit tests around transfer rules and account lifecycle operations.

10 — Suggested Refactoring Sprint Plan

Sprint	Deliverable	Definition of done
S1	Repository structure	src/main/java, src/test/java, db/data, README, .gitignore

S2	Domain model	Account, Customer, Transaction classes compile independently
S3	Service layer	Login/create/transfer/close rules live outside UI code
S4	Persistence adapter	All data access is behind repository interfaces
S5	Testing	Transfer validation and account lifecycle covered by JUnit
S6	Security cleanup	Passwords hashed; sample data removed from public repo

11 — Evaluation Scorecard

Dimension	Current state	Comment
Functionality	Strong for scope	Core banking journey is represented
Readability	Good	Feature classes are easy to locate
OOP demonstration	Good	Interfaces + class decomposition are visible
Persistence design	Basic	Useful for learning, weak for reliability
Security	Weak	Plaintext credentials and personal data
Scalability	Low	Text-file model and tight coupling limit growth
Extensibility	Moderate	Good starting point for a structured refactor

12 — Final Take

The project succeeds at what a second-year OOP exercise should do: it turns abstract programming concepts into a recognisable application with state, validation, files and business rules. The most valuable next step is not simply “adding more features.” It is separating concerns so the same banking idea can survive a change of storage engine, interface, or security model.

Bottom line

Keep the banking workflow. Upgrade the architecture. That turns a classroom implementation into a portfolio-ready Java project without losing the clarity of the original.

Source & Review Basis

Primary repository: <https://github.com/Vedantt-Patel/Bank-Management-System-OOP-JAVA>

Reviewed implementation files include Main.java, Login.java, Creation.java, BalanceInquiry.java, AccountDetails.java, Transaction.java, BankStatement.java, Deletion.java, and the repository README.